

Synchronous vs Asynchronous Apex



Synchronous va asynchronous apex in table

Here's a clear comparison of **Synchronous vs Asynchronous Apex** in Salesforce in a **table format**:

Feature	Synchronous Apex	Asynchronous Apex
Definition	Code that runs in real-time	Code that runs in the background
Execution	Executed immediately	Queued and executed later
User Experience	User waits for execution to complete	User doesn't wait; continues interaction
Examples	Triggers, Controllers, Batchable Web Calls	Future Methods, Queueable, Batch Apex, Schedulable Apex
Governor Limits	Lower limits (strict)	Higher limits (more lenient)
Use Case	Immediate processing like validations or DML	Long-running tasks like callouts, data processing
Transaction Boundaries	Part of current transaction	Runs in separate transaction
Order of Execution	Predictable	Not guaranteed
Callouts to External APIs	Not allowed directly in triggers	Allowed (e.g., in @future or Queueable)
Debugging	Easier to debug (inline execution)	Harder due to background processing
Example Syntax	<code>public void methodName() {}</code>	<code>@future, implements Queueable, implements Batchable</code>

Let me know if you want a **visual diagram** or code examples too.



With governor limits

Absolutely—let's expand the table with **Governor Limits** so you can clearly see the difference:

Feature	Synchronous Apex	Asynchronous Apex
Definition	Code that runs in real time	Code that runs in the background
Execution	Immediately executed	Queued and executed later
User Experience	User waits for completion	User doesn't wait; work happens in the background
Examples	Triggers, Controllers, Validation Rules	@future methods, Queueable Apex, Batch Apex, Schedulable Apex
Transaction Boundaries	Same transaction	Separate transaction
Order of Execution	Predictable	Not guaranteed

Feature	Synchronous Apex	Asynchronous Apex
Callouts to External APIs	Not allowed directly in triggers	Allowed (e.g., @future(callout=true))
Debugging	Easier to debug (inline execution)	Harder due to asynchronous nature
Governor Limits	Lower Limits: - 100 SOQL queries - 150 DML statements - Heap size: 6 MB - CPU time: 10,000 ms - 10 callouts - Max records retrieved by SOQL: 50,000	Higher Limits: - 200 SOQL queries - 150 DML statements - Heap size: 12 MB - CPU time: 60,000 ms - 10 callouts (in future/queueable) - Max records retrieved by SOQL: 50,000
Use Case	Short, fast processing in user context	Large data operations, callouts, scheduled tasks

Key Differences in Governor Limits

✓ SOQL Queries:

- Synchronous: **100**
- Asynchronous: **200**

✓ Heap Size:

- Synchronous: **6 MB**
- Asynchronous: **12 MB**

✓ CPU Time:

- Synchronous: **10,000 ms**
- Asynchronous: **60,000 ms**

✓ Callouts:

- Both: Max **10**, but asynchronous Apex is the recommended approach for callouts.

If you'd like, I can also give you:

- Code snippets for each type
- A deeper look at each limit
- Specific examples (like Queueable vs Batch Apex)